

Vocabulary as Moat

Microsoft DOS and the Politics of Proprietary Convention

Flyxion
Independent Researcher

2026

Contents

1	From CP/M to PC-DOS	1
1.1	86-DOS and the CP/M Mandate	1
1.2	The Switch and the Slash	2
1.3	No Directories, No Conflict	3
1.4	The Argument This Sets Up	3

Chapter 1

From CP/M to PC-DOS

The vocabulary that would come to distinguish Microsoft's operating systems from the Unix tradition for the next four decades was not, in its origin, a Microsoft invention. It was an inheritance, and an accidental one at that. Understanding this inheritance matters for the argument of this book, because the later chapters will claim that Microsoft entrenched and extended a set of naming conventions in order to build a self-contained, non-transferable ecosystem. That claim is strongest once it is clear that the earliest divergences from Unix were not made for that purpose at all. They were made for reasons of compatibility with a much smaller and more parochial system: Digital Research's CP/M.

1.1 86-DOS and the CP/M Mandate

When Tim Paterson wrote 86-DOS at Seattle Computer Products in 1980, his design brief was not to build an alternative to Unix. Unix was, at that point, a research and minicomputer operating system with little presence in the microcomputer market Paterson was writing for. His actual constraint was narrower and more commercial: 86-DOS needed to run software that had been written for CP/M, the dominant operating system for 8-bit microcomputers, on the new Intel 8086 processor for which no CP/M port yet existed. Digital Research was slow to deliver CP/M-86, and Seattle Computer Products wanted an operating system it could ship immediately with its 8086 boards. Paterson's solution was to write an operating system whose system calls and command syntax mirrored CP/M closely enough that CP/M source code could be mechanically translated to run under it.

This single fact, the decision to clone CP/M's interface rather than design a new one, is the origin point for nearly every vocabulary divergence this book will examine. CP/M's command-line conventions were not derived from any deep philosophy of operating system design. They were the idiosyncratic choices of Gary Kildall's small team at Digital Research, made in the mid-1970s for an operating system meant to fit in a few kilobytes of memory alongside 8080 assembly tooling. When Paterson copied CP/M's interface into 86-DOS, and when Microsoft acquired 86-DOS in 1981 and licensed it to IBM as PC-DOS, these idiosyncrasies were not reconsidered. They were preserved, because preservation was the entire point of the exercise. Compatibility with CP/M software and CP/M-trained users was the product being sold.

It is worth pausing on how different this origin story is from Unix's own. Unix's command vocabulary, terse to the point of opacity (`cp`, `mv`, `rm`, `ls`), emerged at Bell Labs from a research culture that valued economy of keystrokes on slow teletypes and a philosophy, later articulated explicitly by Doug McIlroy and others, of small composable tools connected by pipes. Unix's terseness was a design position, argued for and defended. CP/M's conventions, and by inheritance DOS's, were not argued for. They were simply what a small commercial team had settled on for a resource-constrained hobbyist machine, and what a second small commercial team later found it convenient not to change.

1.2 The Switch and the Slash

Among the conventions Paterson inherited was CP/M's use of the forward slash to introduce command options. A CP/M user running the PIP utility to copy files with verification might type `PIP B:=A:*. *[V]`, and other CP/M programs used a bare slash before a letter to select a mode or option, a convention that traces back further still, to the option-flagging habits of DEC operating systems like RT-11 and RSX-11 that influenced Kildall's own background at Digital Equipment Corporation before he founded Digital Research. The slash-prefixed option was, in other words, already a secondhand convention by the time it reached CP/M, and a thirdhand one by the time it reached DOS.

This detail matters because it complicates any simple story in which Microsoft chose the slash to differentiate itself from Unix. Unix's dash-prefixed option (`ls -l`, `cp -r`) was itself a somewhat later standardization; early Unix commands were inconsistent about option syntax before the convention hardened over the 1970s. DOS's slash and Unix's dash are best understood not as competing proposals in a single debate but as parallel outcomes of two lineages, DEC-and-CP/M on one side, Bell Labs on the other, that had already diverged before either PC-DOS or a mature Unix option-parsing convention existed. What Microsoft did was not invent the slash. What Microsoft did, and what later chapters will treat as the real object of political interest, was decline every subsequent opportunity to reconsider it, even as the rationale for keeping it disappeared.

DOS did not call its slash-prefixed options "flags," the term of art that had by then become standard in the Unix world and in the broader Bell Labs-descended tradition of documentation. It called them switches. The word was already present in CP/M documentation and in the DEC minicomputer manuals behind it, evoking a physical toggle on a piece of hardware rather than a bit set in a parsed argument vector. Whether this word choice carried any of the deliberate branding intent that this book will later argue characterizes later Microsoft naming decisions is doubtful for Chapter 1's period; "switch" most likely traveled into DOS documentation simply because it was already the word CP/M and DEC manuals used, and nobody at Microsoft or IBM had reason in 1981 to replace inherited vocabulary that their target audience, largely CP/M refugees, already understood. The word's persistence through every subsequent DOS revision, long after that audience had turned over, is a different matter, and one this book returns to in Chapter 3.

1.3 No Directories, No Conflict

The absence of any collision between the slash-as-switch convention and a slash-as-path-separator convention in early DOS was not a matter of foresight. It was a matter of DOS 1.0, like the CP/M it cloned, having no hierarchical filesystem at all. CP/M organized files into flat, numbered “user areas” on a per-drive basis; there was no notion of nested directories, and therefore no need for any character to separate path components. DOS 1.0 for the IBM PC, released in 1981, inherited this flat structure along with everything else. A file was addressed by a drive letter and a filename, full stop. There was no path syntax to design, and consequently no reason for the slash’s role as a switch prefix to come under any pressure.

This changed in March 1983, with the release of PC-DOS 2.0. IBM’s PC XT, the hardware DOS 2.0 was built to support, shipped with a 10-megabyte hard disk, a capacity at which a flat file namespace becomes genuinely unworkable; users needed to organize hundreds of files into groups. Microsoft’s solution was to add a hierarchical directory tree, and the design of that tree owed an explicit debt to Unix. Microsoft’s own engineers, several of whom had worked on the company’s Xenix port of Unix for the 8086 in the same period, have described DOS 2.0’s directory structure as directly modeled on Unix’s, down to the use of `.` and `..` as entries denoting the current and parent directory. In this one respect, contrary to the entire pattern this book documents elsewhere, Microsoft looked at the Unix solution to a real problem and adopted it rather than inventing a parallel proprietary structure.

But adoption could not be complete, because the slash was already spoken for. Years of DOS 1.x documentation, and more importantly years of already-shipped software that parsed command lines expecting a slash to introduce a switch, meant that using the slash as Unix does, to separate path components, would have broken existing programs the moment a user typed a switch after a pathname or a pathname where a switch was expected. Microsoft’s engineers resolved the conflict by choosing the backslash, a character with no prior meaning in DOS’s command syntax and, at the time, no ASCII keyboard prominence to rival its unshifted cousin, to serve as the path separator instead. This is the origin of the backslash that would go on to define the visual signature of DOS and Windows pathnames (`C:\DOS\COMMAND.COM`) for the following four decades, and it is worth stating plainly what kind of decision it was. It was not a decision to differ from Unix as a matter of policy. It was a decision forced by an earlier, unrelated decision, itself inherited from CP/M, that had already claimed the character Unix used.

1.4 The Argument This Sets Up

The point of recounting this chain of inheritance in such detail is not antiquarian. It is to establish, before this book’s polemical argument begins in earnest, the distinction between path dependence and design intent. Nobody at Microsoft in 1981 sat down and decided that DOS should speak a different language than Unix in order to trap its users inside a proprietary vocabulary. The slash was CP/M’s before it was DOS’s, and CP/M’s before that it was DEC’s. The backslash was a patch, not a proposal. If the story ended in 1983, this would be a much shorter and much less interesting book: a footnote about legacy con-

straints in early microcomputer operating systems, of a kind that could be told about almost any software lineage.

The story does not end in 1983. What makes DOS a subject worth a monograph, rather than a paragraph in a history of CP/M, is what Microsoft did with this inherited vocabulary over the following decade, as the original compatibility rationale for each of these choices quietly expired while the choices themselves did not. CP/M software compatibility, the entire reason 86-DOS existed in its particular shape, was commercially irrelevant by the mid-1980s; DOS's user base had long since become native to DOS rather than migrating from CP/M. Unix, meanwhile, was no longer a research curiosity but an increasingly visible alternative, one that Microsoft's own Xenix division understood well enough to imitate selectively, as the directory tree shows. By the time of MS-DOS 5.0 and 6.0 in the early 1990s, the manuals excerpted throughout this book, there was no CP/M-compatibility reason left to call an option a switch, to prefix it with a slash, to spell `copy` rather than write `cp`, or to keep the drive letter as the unit of storage addressing rather than adopting something closer to a unified filesystem tree. The infrastructure of inheritance had become, by then, a matter of choice sustained rather than a constraint endured. Chapter 2 takes up that shift directly, examining how the path-separator problem, born of accident in 1983, became a template for a much more deliberate pattern of vocabulary divergence in the decade that followed.